

Chetana (चेतना) — Android App PRD

Version: 2.0 — Final (post-audit) **Platform:** Android (Kotlin + Jetpack Compose) **Backend:** AWS Lambda + API Gateway (api_contracts.md) **Tagline:** *Awaken to your health*

1. Vision

Chetana is an AI-powered family health companion that lets Indian families snap photos of lab reports, extract structured data using Amazon Nova, and converse with their own medical records in their preferred language. The mobile app is the only user-facing surface — it must be polished enough to win a hackathon and simple enough to build solo in 2–3 days.

2. Design Philosophy

2.1 Core principles

- **Calm, not clinical** — health apps often feel sterile or scary. Chetana should feel like a knowledgeable friend, not a hospital dashboard.
- **Family-first** — the mental model is a family, not an individual patient. Every screen makes it easy to switch context between members.
- **Show, don't list** — colour-coded status, trend lines, and severity chips over raw tables wherever possible.
- **Progressive disclosure** — summary first, detail on tap. Never overwhelm with data.
- **Indian context** — support Hindi/vernacular text, Indian name conventions, design for varying literacy levels.

2.2 Tone

Warm, clear, non-alarmist. Even "urgent" insights should be framed as "here's what to pay attention to" — never "you are in danger."

3. Colour System

3.1 Brand palette

Token	Hex	Usage
brand-deep	 #1a1145	Splash/header gradients

Token	Hex	Usage
brand-mid	#2d1f6e	Header gradient end, CTA background
brand-purple	#7F77DD	Primary interactive colour, links, active states
brand-purple-light	#EEEDFE	Purple tinted backgrounds, selected states
brand-forest	#0F6E56	Success, normal lab values, positive trends
brand-forest-light	#E1F5EE	Normal value backgrounds

3.2 Semantic colours — lab values (non-negotiable)

State	Background	Text	Usage
Normal (N)	#E1F5EE	#0F6E56	Within reference range
High (H)	#FCEBEB	#A32D2D	Above normal high
Low (L)	#E6F1FB	#185FA5	Below normal low
Critical High (HH)	#E24B4A	#FFFFFF	Prompt medical attention
Critical Low (LL)	#378ADD	#FFFFFF	Prompt medical attention

3.3 Insight severity colours

Severity	Left border	Background	Text
Urgent	#E24B4A	#FCEBEB	#A32D2D
Attention	#EF9F27	#FAEEDA	#633806
Informational	#1D9E75	#E1F5EE	#085041

3.4 UI surface colours

Background primary: #FFFFFF

Background secondary: #F8F8F6

Background tertiary: #F1EFE8

Text primary: #1C1C1A

Text secondary: #5F5E5A

Text tertiary: #888780

Border subtle:	rgba(0,0,0,0.08)
Border default:	rgba(0,0,0,0.14)

3.5 Typography

Font: Inter (primary) · Noto Sans Devanagari (Hindi/Marathi)

Display: 28sp / 500 – splash app name

H1: 22sp / 500 – screen titles

H2: 18sp / 500 – section headers

H3: 16sp / 500 – card titles

Body: 15sp / 400 – primary content

Small: 13sp / 400 – secondary content, metadata

Caption: 11sp / 400 – disclaimers, timestamps

Mono: 13sp / 400 – LOINC codes, lab values

3.6 Shape & spacing

Corner radius – small: 8dp (chips, badges, inputs)

Corner radius – medium: 12dp (cards, buttons)

Corner radius – large: 20dp (bottom sheets, header curves)

Corner radius – pill: 100dp (status chips, avatars)

Spacing scale: 4 / 8 / 12 / 16 / 20 / 24 / 32dp

Card padding: 16dp horizontal, 16dp vertical

Screen margin: 16dp horizontal

4. App Architecture

4.1 Tech stack

Language: Kotlin

UI: Jetpack Compose (Material 3)

Architecture: MVVM + Repository pattern

DI: Hilt

Navigation: Compose Navigation (single-activity)

Network: Retrofit 2 + OkHttp + Kotlin Serialization

Images: Coil 3

Camera: CameraX

Charts: Vico (Compose-native)

Storage: DataStore Preferences

Async: Coroutines + StateFlow

4.2 Project structure

```
app/
├─ di/
│   ├── AppModule.kt           # Retrofit, OkHttp, DataStore
│   ├── RepositoryModule.kt    # Bind interfaces to implementations
│   └─ AuthModule.kt          # MockAuthRepository binding
├─ data/
│   ├── api/
│   │   ├── ChetanaApiService.kt # All Retrofit endpoints
│   │   └─ ApiKeyInterceptor.kt  # x-api-key header
│   ├── model/
│   │   ├── Family.kt
│   │   ├── Member.kt           # resourceType: "Patient"
│   │   ├── DiagnosticReport.kt # resourceType: "DiagnosticReport"
│   │   ├── Observation.kt      # resourceType: "Observation"
│   │   ├── InsightCard.kt
│   │   └─ FollowUp.kt
│   ├── repository/
│   │   ├── AuthRepository.kt    # interface
│   │   ├── MockAuthRepository.kt # reads mock_auth.json
│   │   ├── FamilyRepository.kt  # interface
│   │   ├── LiveFamilyRepository.kt # Retrofit
│   │   ├── ReportRepository.kt  # interface + Live impl
│   │   ├── ChatRepository.kt    # interface + Live impl
│   │   ├── InsightRepository.kt # interface
│   │   ├── MockInsightRepository.kt # reads mock_insights.json
│   │   ├── FollowUpRepository.kt # interface
│   │   └─ MockFollowUpRepository.kt # reads mock_followups.json
│   └─ session/
│       └─ SessionManager.kt     # familyId, memberId, accessToken
├─ ui/
│   ├── theme/
│   │   ├── Color.kt
│   │   ├── Type.kt
│   │   ├── Shape.kt
│   │   └─ Theme.kt
│   ├── components/
│   │   ├── ObservationRow.kt
│   │   ├── InsightCard.kt
│   │   ├── MemberAvatar.kt
│   │   ├── SeverityBadge.kt
│   │   ├── ConfidenceBadge.kt
│   │   ├── DisclaimerBar.kt
│   │   └─ LoadingDots.kt
│   └─ auth/
```

```

|   |   ├── SplashScreen.kt
|   |   ├── LoginScreen.kt
|   |   ├── OtpScreen.kt
|   |   └── AuthViewModel.kt
|   ├── family/
|   |   ├── FamilyDashboard.kt
|   |   ├── MemberDetailScreen.kt
|   |   └── FamilyViewModel.kt
|   ├── upload/
|   |   ├── CaptureScreen.kt
|   |   ├── ProcessingScreen.kt
|   |   ├── ResultScreen.kt
|   |   └── UploadViewModel.kt
|   ├── report/
|   |   ├── ReportDetailScreen.kt
|   |   └── ReportViewModel.kt
|   ├── chat/
|   |   ├── ChatScreen.kt
|   |   └── ChatViewModel.kt
|   ├── insights/
|   |   ├── InsightsFeedScreen.kt
|   |   ├── FollowUpsScreen.kt
|   |   └── InsightsViewModel.kt
|   └── settings/
|       └── SettingsScreen.kt
└── util/
    ├── LoincHelper.kt
    ├── SaMDStringHelper.kt
    ├── ChatResponseValidator.kt
    ├── FhirDateFormatter.kt
    └── MockDataLoader.kt
└── assets/
    ├── mock_auth.json
    ├── mock_insights.json
    └── mock_followups.json

```

4.3 Navigation graph

```

NavHost {
    SplashScreen          (no back stack entry)
    LoginScreen
    OtpScreen

    // popUpTo(SplashScreen) { inclusive = true } on OTP success
    MainScaffold (bottom nav: Family | Upload | Chat | Insights) {
        FamilyDashboard
    }
}

```

```
MemberDetailScreen → ReportDetailScreen
CaptureScreen → ProcessingScreen → ResultScreen → (Chat deep link)
ChatScreen          (member switcher in header)
InsightsFeedScreen  (secondary tab: Insights | Follow-ups)
FollowUpsScreen

}

SettingsScreen      (gear icon – reachable from all main tabs)

}
```

5. FHIR R4 Data Models

All field names map directly to FHIR R4 elements.

```
kotlin
```

@Serializable

```
data class DiagnosticReport(  
    val resourceType: String = "DiagnosticReport",  
    val reportId: String,  
    val status: ReportStatus,           // FHIR: status  
    val effectiveDateTime: String,      // FHIR: effectiveDateTime (YYYY-MM-DD)  
    val issued: String? = null,         // FHIR: issued (ISO 8601)  
    val labName: String,                // FHIR: performer[0].display  
    val memberId: String,               // FHIR: subject.reference  
    val memberName: String? = null,  
    val autoDetected: Boolean = false,  
    val totalObservations: Int,  
    val abnormalCount: Int,  
    val extractionConfidence: Float,  
    val observations: List<Observation> = emptyList(),  
    val immediateInsights: List<InsightCard> = emptyList(),  
    val disclaimer: String,  
    val s3Key: String? = null  
)
```

```
enum class ReportStatus { uploading, processing, completed, failed, final, preliminary }
```

@Serializable

```
data class Observation(  
    val resourceType: String = "Observation",  
    val id: String,  
    val status: String = "final",  
    val name: String,                // FHIR: code.text  
    val loincCode: String,           // FHIR: code.coding[0].code  
    val category: String,            // FHIR: category (Blood Sugar, Lipid Pa  
    val value: Double,               // FHIR: valueQuantity.value  
    val unit: String,                // FHIR: valueQuantity.unit  
    val normalLow: Double,           // FHIR: referenceRange[0].low - FROM L  
    val normalHigh: Double,          // FHIR: referenceRange[0].high - FROM L  
    val isAbnormal: Boolean,  
    val interpretation: String,      // FHIR: N / H / L / HH / LL  
    val effectiveDateTime: String,    // FHIR: effectiveDateTime (YYYY-MM-DD)  
    val reportId: String  
)
```

@Serializable

```
data class Member(  
    val resourceType: String = "Patient",  
    val id: String,  
    val name: String,                // FHIR: name[0].text  
    val initials: String,
```

```
    val relationship: String,
    val gender: String? = null,           // FHIR: gender
    val birthDate: String? = null,       // FHIR: birthDate (YYYY-MM-DD)
    val preferredLanguage: String = "en", // FHIR: communication[0].language
    val avatarColor: String
)
```

6. SaMD Rules

6.1 The 5 rules — enforced at every layer

#	Rule	Android enforcement
1	Never generate a diagnosis	Bedrock Guardrails (backend)
2	Never recommend treatment or medication	Bedrock Guardrails (backend)
3	Never say patient "has" a condition	<code>SaMDStringHelper</code> + <code>ChatResponseValidator</code>
4	Always include medical disclaimer	<code>DisclaimerBar</code> — structural, non-dismissible
5	Show extraction confidence	<code>ConfidenceBadge</code> on every Result + ReportDetail screen

6.2 SaMDStringHelper

kotlin


```

object SaMDStringHelper {

    fun interpretationLabel(code: String): String = when(code) {
        "N" -> "Within range"
        "H" -> "Above reference range"
        "L" -> "Below reference range"
        "HH" -> "Critically elevated – seek medical attention"
        "LL" -> "Critically low – seek medical attention"
        else -> "See report"
    }

    fun severityLabel(severity: String): String = when(severity) {
        "urgent" -> "Requires prompt review"
        "attention" -> "Worth monitoring"
        "informational" -> "For your awareness"
        else -> severity
    }

    fun trendDescription(testName: String, direction: String, count: Int): String =
        "Your $testName has been ${if(direction=="up") "rising" else "falling"} " +
        "across $count consecutive reports. Please consult your doctor."

    val disclaimerFull =
        "Chetana does not provide medical advice. All information is for " +
        "informational purposes only. Always consult a licensed healthcare provider."

    val disclaimerShort = "For informational purposes only. Consult your doctor."

    val disclaimerInline = " ⓘ This is not medical advice."

    val extractionDisclaimer =
        "This is an AI-assisted summary of your lab report. " +
        "Always verify with your original report and consult your doctor."
}

```

6.3 ChatResponseValidator

kotlin

```

object ChatResponseValidator {

    private val BLOCKED_PHRASES = listOf(
        "you have diabetes", "you are diabetic",
        "you have hypertension", "you have anaemia", "you are anaemic",
        "i recommend taking", "you should take", "prescribe",
        "diagnosed with", "suffering from", "you are sick"
    )

    fun validate(response: String): ValidationResult {
        val lower = response.lowercase()
        val blocked = BLOCKED_PHRASES.firstOrNull { lower.contains(it) }
        return if (blocked != null)
            ValidationResult.Blocked(
                "I'm not able to answer that directly. " +
                "Please consult your doctor for a diagnosis or treatment recommendation." +
                SaMDStringHelper.disclaimerShort
            )
        else
            ValidationResult.Safe(response)
    }

    sealed class ValidationResult {
        data class Safe(val text: String) : ValidationResult()
        data class Blocked(val fallback: String) : ValidationResult()
    }
}

```

6.4 ConfidenceBadge thresholds

kotlin

```

fun confidenceStyle(confidence: Float) = when {
    confidence >= 0.90 -> ConfidenceStyle("#E1F5EE", "#0F6E56", "High confidence")
    confidence >= 0.70 -> ConfidenceStyle("#FAEEED", "#633806", "Moderate confidence")
    else -> ConfidenceStyle("#FCEBEB", "#A32D2D", "Low confidence - verify")
}

```

If `confidence < 0.70`, render a full amber warning card **above the observation table**:

"AI confidence is below 70%. Some values may be incorrectly extracted. Please verify all results against your original paper report before sharing with your doctor."

6.5 Language rules — applied to every string

✗ Never say

✓ Always say

"You have high glucose"

"Your glucose is above the reference range"

"You are pre-diabetic"

"Your HbA1c is in the pre-diabetic range (5.7–6.4%)"

"3 abnormal"

"3 outside range"

"All normal"

"All within range"

"Critical"

"Requires prompt medical attention"

6.6 DisclaimerBar — structural requirement

kotlin

@Composable

```
fun DisclaimerBar(variant: DisclaimerVariant = DisclaimerVariant.FULL) {  
    val text = when(variant) {  
        DisclaimerVariant.FULL    -> SaMDStringHelper.disclaimerFull  
        DisclaimerVariant.SHORT   -> SaMDStringHelper.disclaimerShort  
        DisclaimerVariant.INLINE  -> SaMDStringHelper.disclaimerInline  
    }  
    Box(  
        modifier = Modifier  
            .fillMaxWidth()  
            .background(Color(0xFFF8F8F6))  
            .border(0.5.dp, Color(0x1A000000), RectangleShape)  
            .padding(horizontal = 16.dp, vertical = 8.dp)  
    ) {  
        Text(text, fontSize = 11.sp, color = Color(0xFF888780),  
            lineHeight = 16.sp, textAlign = TextAlign.Center)  
    }  
}
```

```
enum class DisclaimerVariant { FULL, SHORT, INLINE }
```

Placement rules:

- **FULL** — pinned below input bar on ChatScreen (non-dismissible)
- **SHORT** — sticky footer on ResultScreen and ReportDetailScreen
- **INLINE** — last line of every InsightCard

7. LOINC Helper

kotlin

```
object LoincHelper {
```

```
    data class LoincEntry(val code: String, val displayName: String,  
        val category: String, val unit: String,  
        val normalLow: Double, val normalHigh: Double)
```

```
    val MAP: Map<String, LoincEntry> = mapOf(  
        "2339-0" to LoincEntry("2339-0", "Glucose (Fasting)", "Blood Sugar", "mg",  
        "4548-4" to LoincEntry("4548-4", "HbA1c", "Blood Sugar", "%",  
        "20448-7" to LoincEntry("20448-7", "Insulin (Fasting)", "Blood Sugar", "µI",  
        "2093-3" to LoincEntry("2093-3", "Total Cholesterol", "Lipid Panel", "mg",  
        "18262-6" to LoincEntry("18262-6", "LDL Cholesterol", "Lipid Panel", "mg",  
        "2085-9" to LoincEntry("2085-9", "HDL Cholesterol", "Lipid Panel", "mg",  
        "2571-8" to LoincEntry("2571-8", "Triglycerides", "Lipid Panel", "mg",  
        "718-7" to LoincEntry("718-7", "Hemoglobin", "CBC", "g/",  
        "6690-2" to LoincEntry("6690-2", "WBC", "CBC", "10",  
        "789-8" to LoincEntry("789-8", "RBC", "CBC", "10",  
        "777-3" to LoincEntry("777-3", "Platelets", "CBC", "10",  
        "4544-3" to LoincEntry("4544-3", "Hematocrit", "CBC", "%",  
        "787-2" to LoincEntry("787-2", "MCV", "CBC", "fL",  
        "2160-0" to LoincEntry("2160-0", "Creatinine", "Kidney", "mg",  
        "3094-0" to LoincEntry("3094-0", "BUN", "Kidney", "mg",  
        "33914-3" to LoincEntry("33914-3", "eGFR", "Kidney", "mL",  
        "3084-1" to LoincEntry("3084-1", "Uric Acid", "Kidney", "mg",  
        "1742-6" to LoincEntry("1742-6", "ALT (SGPT)", "Liver", "U/",  
        "1920-8" to LoincEntry("1920-8", "AST (SGOT)", "Liver", "U/",  
        "1975-2" to LoincEntry("1975-2", "Bilirubin Total", "Liver", "mg",  
        "1751-7" to LoincEntry("1751-7", "Albumin", "Liver", "g/",  
        "6768-6" to LoincEntry("6768-6", "ALP", "Liver", "U/",  
        "3016-3" to LoincEntry("3016-3", "TSH", "Thyroid", "µI",  
        "3051-0" to LoincEntry("3051-0", "T3 (Free)", "Thyroid", "pg",  
        "3054-4" to LoincEntry("3054-4", "T4 (Free)", "Thyroid", "ng",  
        "2951-2" to LoincEntry("2951-2", "Sodium", "Electrolytes", "mE",  
        "2823-3" to LoincEntry("2823-3", "Potassium", "Electrolytes", "mE",  
        "17861-6" to LoincEntry("17861-6", "Calcium", "Electrolytes", "mg",  
        "2601-3" to LoincEntry("2601-3", "Magnesium", "Electrolytes", "mg",  
        "1989-3" to LoincEntry("1989-3", "Vitamin D (25-OH)", "Vitamins", "ng",  
        "2132-9" to LoincEntry("2132-9", "Vitamin B12", "Vitamins", "pg",  
        "2498-4" to LoincEntry("2498-4", "Iron", "Vitamins", "µg",  
        "2276-4" to LoincEntry("2276-4", "Ferritin", "Vitamins", "ng",  
    )
```

```
    fun get(code: String): LoincEntry? = MAP[code]
```

```
    fun category(code: String): String = MAP[code]?.category ?: "Other"
```

```
    fun groupByCategory(obs: List<Observation>): Map<String, List<Observation>> =
```

```
obs.groupBy { category(it.loincCode) }  
}
```

8. FHIR Date Handling

kotlin

```
object FhirDateFormatter {  
    private val fhirFmt = DateTimeFormatter.ofPattern("yyyy-MM-dd")  
    private val displayFmt = DateTimeFormatter.ofPattern("d MMM yyyy")  
    private val displayHi = DateTimeFormatter.ofPattern("d MMMM yyyy", Locale("hi",  
  
    fun toDisplay(fhirDate: String, locale: String = "en"): String = runCatching {  
        val d = LocalDate.parse(fhirDate, fhirFmt)  
        if (locale == "hi") d.format(displayHi) else d.format(displayFmt)  
    }.getOrElse(fhirDate)  
  
    fun toFhir(displayDate: String): String = runCatching {  
        LocalDate.parse(displayDate, displayFmt).format(fhirFmt)  
    }.getOrElse(displayDate)  
}
```

Rule: Every date stored in a data class uses `YYYY-MM-DD`. Every date shown in the UI passes through `FhirDateFormatter.toDisplay()`. Never store or pass display-formatted dates to API calls.

9. Mock Data Strategy

9.1 Which endpoints need mock

Endpoint	Reason	Mock file
POST /auth/login	Cognito removed for hackathon	mock_auth.json
POST /auth/verify-otp	Cognito removed for hackathon	mock_auth.json
GET /insights	Lambda 3 not wired	mock_insights.json
PATCH /insights/{id}	Lambda 3 not wired	local state only
GET /followups	Lambda 3 not wired	mock_followups.json
PATCH /followups/{id}	Lambda 3 not wired	local state only

All other endpoints (`families`, `members`, `reports`, `observations`, `chat`) use real API.

9.2 Asset placement

```
app/src/main/assets/  
├─ mock_auth.json  
├─ mock_insights.json  
└─ mock_followups.json
```

9.3 MockDataLoader

```
kotlin  
  
object MockDataLoader {  
    fun <T> load(context: Context, fileName: String, type: Type): T {  
        val json = context.assets.open(fileName)  
            .bufferedReader().use { it.readText() }  
        return Gson().fromJson(json, type)  
    }  
}
```

9.4 Repository implementations

```
kotlin
```

```

class MockAuthRepository(private val ctx: Context) : AuthRepository {
    private val data by lazy { MockDataLoader.load<JsonObject>(ctx, "mock_auth.json")

    override suspend fun signIn(email: String, password: String): Result<Unit> {
        delay(1200)
        val valid = data["validCredentials"].asJsonObject
        return if (email == valid["email"].asString && password == valid["password"]
            Result.success(Unit)
        else Result.failure(Exception("Invalid email or password. Please try again."
    }

    override suspend fun verifyOtp(code: String): Result<String> {
        delay(1000)
        return if (code.length == 6)
            Result.success(data["otpSuccessResponse"].asJsonObject["familyId"].asStr
        else Result.failure(Exception("Incorrect code. Please try again."))
    }
}

class MockInsightRepository(private val ctx: Context) : InsightRepository {
    private val data by lazy { MockDataLoader.load<InsightsResponse>(ctx, "mock_insi

    override suspend fun getInsights(familyId: String, memberId: String): Result<Ins
        delay(300)
        return Result.success(data)
    }

    override suspend fun markAsRead(insightId: String): Result<Unit> = Result.succes
}

class MockFollowUpRepository(private val ctx: Context) : FollowUpRepository {
    private val data by lazy { MockDataLoader.load<FollowUpsResponse>(ctx, "mock_fol

    override suspend fun getFollowUps(familyId: String, memberId: String): Result<Fo
        delay(300)
        return Result.success(data)
    }

    override suspend fun updateStatus(followUpId: String, status: String): Result<Un
}

```

9.5 Hilt bindings — one line swap per repository

kotlin


```

@Module @InstallIn(SingletonComponent::class)
object RepositoryModule {

    @Provides @Singleton
    fun provideAuthRepository(@ApplicationContext ctx: Context): AuthRepository =
        MockAuthRepository(ctx) // → CognitoAuthRepository() post-hackatho

    @Provides @Singleton
    fun provideInsightRepository(@ApplicationContext ctx: Context): InsightRepository =
        MockInsightRepository(ctx) // → LiveInsightRepository(apiService) pos

    @Provides @Singleton
    fun provideFollowUpRepository(@ApplicationContext ctx: Context): FollowUpRepository =
        MockFollowUpRepository(ctx) // → LiveFollowUpRepository(apiService) po
}

```

10. Screens — Detailed Specification

10.1 Splash Screen

Layout: Full screen gradient (#1a1145 → #2d1f6e → #0F6E56). Centred: 🌿 logo mark (88dp rounded square), "चेतना" (32sp/500), "CHETANA" (14sp letter-spaced), tagline "Awaken to your health" (17sp italic), "Get Started →" button, "Powered by Amazon Bedrock · Nova AI" (11sp).

Logic: Check DataStore (isLoggedIn). If true → FamilyDashboard. If false → auto-advance after 2s → LoginScreen.

10.2 Login Screen

Layout: 200dp gradient header with logo. Form: email input, password input (show/hide toggle), demo credentials hint card (tappable to auto-fill), "Continue →" CTA button (gradient (#7F77DD → #534AB7)), divider, Google + Phone OTP secondary buttons (show "post-hackathon" toast), sign up link.

Demo credentials: demo@chetana.health / demo1234

Mock: MockAuthRepository.signIn() with 1200ms simulated delay.

Post-hackathon: Replace with Amplify.Auth.signIn().

10.3 OTP Screen

Layout: 170dp gradient header. 6 individual `BasicTextField` boxes ($44\text{dp} \times 52\text{dp}$, 8dp gap). Auto-focus next on digit entry. Backspace from empty box focuses previous. Active box: purple border + tinted fill. "Verify & Enter →" CTA. "Resend" text link.

Mock: Accept any 6 digits. `MockAuthRepository.verifyOtp()` with 1000ms delay.

Post-hackathon: Replace with `Amplify.Auth.confirmSignIn(code)`.

10.4 Family Dashboard (Tab 1)

Top bar: 🌿 logo + "चेतना" left. Language chip + settings gear right (gear navigates to `SettingsScreen`).

Family Health Score card: Computed client-side: $\frac{100 - (\text{totalAbnormals} / \text{totalObservations} * 100)}{}$. Shows score (40sp), "Based on N reports · M members", trend arrow vs last month.

Member scroll rail: Horizontal scroll. Each card (130dp wide): initials avatar (colour from name hash), first name, relationship chip, "X outside range" (red) or "All within range" (green), last report date. Tap → `MemberDetailScreen`.

Recent reports: Last 3 reports across all members. Each row: avatar + name, lab name + date, abnormal count chip.

FAB: Camera icon, brand purple, bottom-right. → `CaptureScreen`.

API: `GET /families/{fid}/members`

10.5 Member Detail Screen

Top bar: Back + member name + relationship. Member avatar right.

Tab row (4 tabs): Reports | Trends | Insights | Follow-ups

Reports tab: List sorted newest first. Each card: date, lab name, test count, "X outside range" / "All within range" badge. Tap → `ReportDetailScreen`. API: `GET /reports`

Trends tab: Vico `LineChart` per LOINC category. Each chart: test name + `LOINC {code}` subtitle, current value + trend arrow, green reference band between `normalLow` and `normalHigh`, red data points for abnormal values. Caption: "Ref band: X-Y (lab's range)". Mock data (3 months).

Insights tab: Filtered `InsightCards` for this member. Mock data.

Follow-ups tab: `FollowUpCards` for this member. Mock data.

10.6 Report Detail Screen

Top bar: Back + "Report detail" + share icon.

Layout:

- Zoomable image area (200dp default, pinch to zoom via PhotoView). Dark background with  placeholder + "Pinch to zoom".
- `ConfidenceBadge` below image.
- If `confidence < 0.70`: amber warning card — "AI confidence is below 70%. Verify all results against your original paper report."
- FHIR Patient match banner (purple tinted).
- Section header: "Lab results · FHIR Observation" + "{N} outside range" count.
- Observation rows (see ObservationRow component).
- Extraction disclaimer card (purple tinted, `SamDStringHelper.extractionDisclaimer`).
- Sticky bottom: "Chat about this report →" (primary) + "Delete" (outline) buttons.
- `DisclaimerBar(variant = SHORT)`.

API: Data comes from UploadViewModel/ResultScreen — no additional API call needed. Delete calls `DELETE /reports/{id}`.

10.7 Capture Screen (Tab 2)

Layout: Full-screen CameraX preview. Corner guide overlay (4 L-shaped corners, `rgba(255, 255, 255, 0.8)`, 28dp long, 2dp). Hint text: "Position the full report within the frame".
Top bar: × dismiss + title. Bottom bar: gallery icon | 68dp capture button (brand purple) | document/PDF icon.

Flow: Capture/pick → `POST /reports/upload-url` → `PUT` to S3 → navigate to ProcessingScreen.

10.8 Processing Screen

Layout: Centred.  logo mark (80dp). "Nova is reading your report" (20sp/500). Cycling status messages (fade in/out every 2.2s): "Reading report header..." → "Identifying test values..." → "Mapping to LOINC codes..." → "Cross-referencing your history..." → "Generating insights...". 3 animated dots. "This takes 10–20 seconds" caption. "You can navigate away ↗" dismiss link.

Logic: Poll `GET /reports/{id}/status` every 3s. `completed` → ResultScreen. `failed` → error state + retry. Navigate away → WorkManager job continues polling → notification on completion.

10.9 Result Screen

Layout:

- Green success banner (✓ "Report extracted successfully" + lab + date).
- `ConfidenceBadge`. Low-confidence warning card if < 0.70 .
- FHIR Patient match banner.
- "Lab results · FHIR Observation" header + "{N} outside range" / "{N} within range" count.
- ObservationRow list (stagger animation: 80ms delay per row).
- "AI insights" section with up to 3 InsightCards.
- Extraction disclaimer card.
- Sticky: "Chat about this report →" button + `DisclaimerBar(SHORT)`.

Observation stagger: Rows animate in with fade + slide-up, 80ms between each. Abnormal rows pulse red on entry.

10.10 Chat Screen (Tab 3)

Top bar: Back + member avatar + member name + "Tap to switch" chevron. Tapping opens member picker dropdown (all family members, purple checkmark on active, switching resets conversation). Language chip right.

Messages: User bubbles right (brand purple, sharp bottom-right corner). Assistant bubbles left (white card, sharp bottom-left corner, 🌿 logo 26dp). Cited observation chips inside assistant bubbles (teal, `name · loincCode` monospace). Typing indicator (3 bouncing dots).

Suggested questions rail: Shown only when conversation is empty. Horizontal scroll chips: "How is my glucose?", "Summarize last report", "What should I test next?", "Is my HbA1c improving?", "How is my cholesterol?". Disappears after first message.

Input bar: 🎤 mic icon left (voice — "coming soon" toast for non-EN/HI). `BasicTextField` with placeholder. Send button (activates on non-empty input).

Chat response flow:

1. Send message to `POST /chat`
2. Pass response through `ChatResponseValidator.validate()`
3. If `Blocked` → show fallback: "I'm not able to answer that directly. Please consult your doctor."
4. If `Safe` → display response with cited observation chips

Disclaimer: `DisclaimerBar(FULL)` always visible below input — non-dismissible, non-scrollable.

Session management: `currentSessionId: String` in ViewModel, initialised empty. Updated from every response. Passed in every request body.

API: `POST /families/{fid}/members/{mid}/chat`

10.11 Insights Feed (Tab 4)

Top bar: "Insights" title. Unread count badge on tab icon (red pill, live count).

Secondary tab row: "Insights | Follow-ups" — underline indicator, full width.

Insights tab:

- Filter chip row: "All · Urgent · Attention · Info" — filtered client-side.
- "Today · {N} unread" section header.
- InsightCard list.

Follow-ups tab: FollowUpCard timeline list.

Mock: `MockInsightRepository` + `MockFollowUpRepository`.

10.12 InsightCard Component

Props: `insightId`, `severity`, `title`, `summary`, `memberName`,
`citedObservations[]`, `suggestedAction`, `read`, `generatedAt`

Layout:

- 4dp left border (severity colour)
- Unread: `rgba(127,119,221,0.03)` background, stronger border
- Top row: `SaMDStringHelper.severityLabel()` badge + member chip + timestamp
- Title (16sp/500) — uses pattern language, never condition assertion
- Summary (12sp, 2-line clamp, expandable on tap)
- Cited obs chips: `"{name} · {loincCode}"` monospace, teal
- Suggested action (11sp italic, "↳ ...")
- Bottom row: `SaMDStringHelper.disclaimerInline` (left) + "Mark read ✓" button (right, only if unread)

"Mark read ✓" triggers `PATCH /insights/{insightId}` (mock: local state update only).

10.13 FollowUpCard Component

Props: followUpId, testName, loincCode, reason, suggestedDate, memberName, status

Layout:

- Test name (16sp/500) + LOINC code badge (grey monospace pill, top right)
- Reason (12sp italic, muted)
- Suggested date (coloured: amber if this week, grey otherwise)
- Accept ✓ (teal fill) + Dismiss × (grey outline) – only if status=pending
- "Accepted ✓" chip if accepted, card fades to 50% opacity if dismissed

Triggers `PATCH /followups/{followUpId}` (mock: local state update only).

10.14 Settings Screen

Sections:

Language & voice Language picker grid: English, हिंदी, मराठी, தமிழ், বাংলা, తెలుగు, ಕನ್ನಡ, ગુજરાતી. Selected = brand purple fill. Voice toggle (EN + HI only; disabled for others with caption).

Family Member list with avatar, name, relationship, "FHIR Patient/{id}" subtitle. Edit + delete per member. "Add member" dashed button.

Data standards

FHIR R4	– DiagnosticReport, Observation, Patient
LOINC	– laboratory test coding (loinc.org)
HL7 v3	– interpretation codes N/H/L/HH/LL
Class I SaMD	– informational, non-diagnostic

About App version, full disclaimer text, "Powered by Amazon Bedrock · Nova AI".

11. UX Patterns

11.1 Loading states

- Shimmer skeleton matching layout shape. 200ms fade in, 1.5s loop.
- Never show blank screen — always shimmer or cached data.

11.2 Empty states

- No reports: "Upload your first report to get started" + FAB hint.

- No insights: "Upload a report to generate insights."
- No follow-ups: "No pending follow-ups."

11.3 Error states

- Red banner top of screen, descriptive message, retry button.
- Chat offline: "You're offline" in input area, send disabled.

11.4 Animations

- Observation row stagger: 80ms delay between rows, fade + slide-up.
- FAB: scale spring on dashboard mount.
- Processing screen: pulse loop on logo mark.
- Insight cards: slide up with stagger on first load.
- Tab badge: scale bounce on count increment.

11.5 Haptics

- Camera capture: medium impact.
- OTP verified: success notification feedback.
- Chat send: light impact.
- Observation tap: light selection.

11.6 Accessibility

- All interactive elements: minimum 48dp touch target.
 - Colour never conveys meaning alone — always paired with text or icon.
 - Content descriptions on all images, icons, and status indicators.
 - System font size scaling supported.
-

12. Demo Script (3 scenarios)

Scenario 1 — Snap & Extract (WOW #1)

1. Splash → login (tap demo fill) → OTP (any 6 digits) → Family Dashboard.
2. Show Sharma family, health score, member cards.
3. Tap FAB → camera → capture lab report.
4. Processing screen: cycling Nova messages.
5. Result: observations animate in, abnormals highlighted red.

- 6. Point to "LOINC 2339-0" under Glucose. Say: "Every test maps to a LOINC code."
- 7. Show confidence badge. Say: "AI knows when it's uncertain."

Scenario 2 — Chat (WOW #2)

- 1. From result screen → "Chat about this report →".
- 2. Switch member to Sunita via header dropdown.
- 3. Switch back to Rahul. Type "Is my HbA1c improving?".
- 4. Show response: pattern language, cited obs chips, disclaimer bar.
- 5. Type "do I have diabetes?" → show blocked response. Say: "Bedrock Guardrails in action."
- 6. Show DisclaimerBar. Say: "Structural — cannot be dismissed — SaMD requirement."

Scenario 3 — Insights & Follow-ups

- 1. Insights tab → red badge. Show urgent card "Glucose trending upward."
- 2. Expand card. Point to "Glucose · 2339-0" chip. Point to " ⓘ This is not medical advice."
- 3. Tap "Mark read ✓". Badge decrements.
- 4. Switch to Follow-ups tab. Show "HbA1c" card. Tap Accept.
- 5. Settings → Data standards block. Say: "FHIR R4, LOINC, HL7 throughout."

13. Post-Hackathon Migrations

Area	Hackathon	Post-hackathon
Auth	MockAuthRepository	AWS Amplify + Cognito User Pool
OTP	Any 6 digits	Cognito SMS/email OTP
Insights	MockInsightRepository	GET /insights + PATCH /insights/{id}
Follow-ups	MockFollowUpRepository	GET /followups + PATCH /followups/{id}
Voice	"Coming soon" toast	Nova 2 Sonic via Lambda 4 WebSocket
Social login	Error toast	Cognito federated (Google, Apple)
Push notifications	Pull on app open	SNS + FCM
PHI security	Demo only	KMS + VPC + HIPAA BAA

14. Build Order (3-day sprint)

Day 1 — Foundation + Auth + Upload flow

- ☐ Project setup: Hilt, Retrofit, Navigation, ChetanaTheme
- ☐ `mock_auth.json`, `mock_insights.json`, `mock_followups.json` in assets
- ☐ All data models (FHIR-aligned field names)
- ☐ `SaMDStringHelper`, `ChatResponseValidator`, `LoincHelper`, `FhirDateFormatter`
- ☐ `MockDataLoader` + all 3 mock repositories
- ☐ Hilt module bindings
- ☐ `SessionManager`
- ☐ SplashScreen + LoginScreen + OtpScreen
- ☐ FamilyDashboard (real API)
- ☐ CaptureScreen (CameraX)
- ☐ ProcessingScreen + polling
- ☐ ResultScreen with observation stagger

Day 2 — Chat + Insights + Member screens

- ☐ ChatScreen with sessionId + member switcher
- ☐ `ChatResponseValidator` integration in ChatViewModel
- ☐ InsightsFeedScreen with secondary tabs (mock data)
- ☐ FollowUpsScreen (mock data)
- ☐ MemberDetailScreen (4 tabs)
- ☐ ReportDetailScreen (PhotoView + observation table)
- ☐ Trends charts (Vico, mock data)

Day 3 — Polish + Demo prep

- ☐ All loading/skeleton states
- ☐ All empty states
- ☐ Error handling + retry
- ☐ Animation pass (stagger, FAB spring, processing pulse)
- ☐ Haptics
- ☐ SettingsScreen + language picker
- ☐ End-to-end demo test (all 3 scenarios)
- ☐ Record backup video

15. Pre-Demo Compliance Checklist

SaMD

- ☐ No screen says "you have [condition]"
- ☐ No screen says "take [medication/supplement]"
- ☐ "X outside range" not "X abnormal" everywhere
- ☐ `DisclaimerBar(FULL)` pinned in ChatScreen
- ☐ `DisclaimerBar(SHORT)` on ResultScreen and ReportDetailScreen
- ☐ `SaMDStringHelper.disclaimerInline` on every InsightCard
- ☐ `ConfidenceBadge` on ResultScreen and ReportDetailScreen
- ☐ Low confidence (< 0.70) renders amber warning card
- ☐ Critical HH/LL rows show "seek medical attention" banner
- ☐ `ChatResponseValidator` running on every assistant message
- ☐ Blocked state shows visible fallback message

FHIR

- ☐ All data models have `resourceType` field
- ☐ Field is `effectiveDateTime` not `date` in Kotlin models
- ☐ `issued: String?` present in `DiagnosticReport`
- ☐ LOINC codes visible in observation rows (monospace grey)
- ☐ LOINC in cited obs chips: "{name} · {loincCode}"
- ☐ LOINC on Trends chart subtitles
- ☐ LOINC badge on FollowUpCards
- ☐ All dates stored `YYYY-MM-DD`, displayed via `FhirDateFormatter`
- ☐ Settings/About shows FHIR R4, LOINC, HL7 v3, Class I SaMD

This is the single source of truth for the Chetana Android build. Version 2.0 incorporates all audit findings. The mockup artifact reflects this specification exactly.